

The Quest for Consistency

A Deep Dive into Database Concurrency Control

In a world of simultaneous database requests, how do we protect the integrity of our data? This is the fundamental challenge of concurrency control—a system for imposing order on concurrent operations to ensure they execute correctly.



The Peril of Parallelism: Why We Need Control

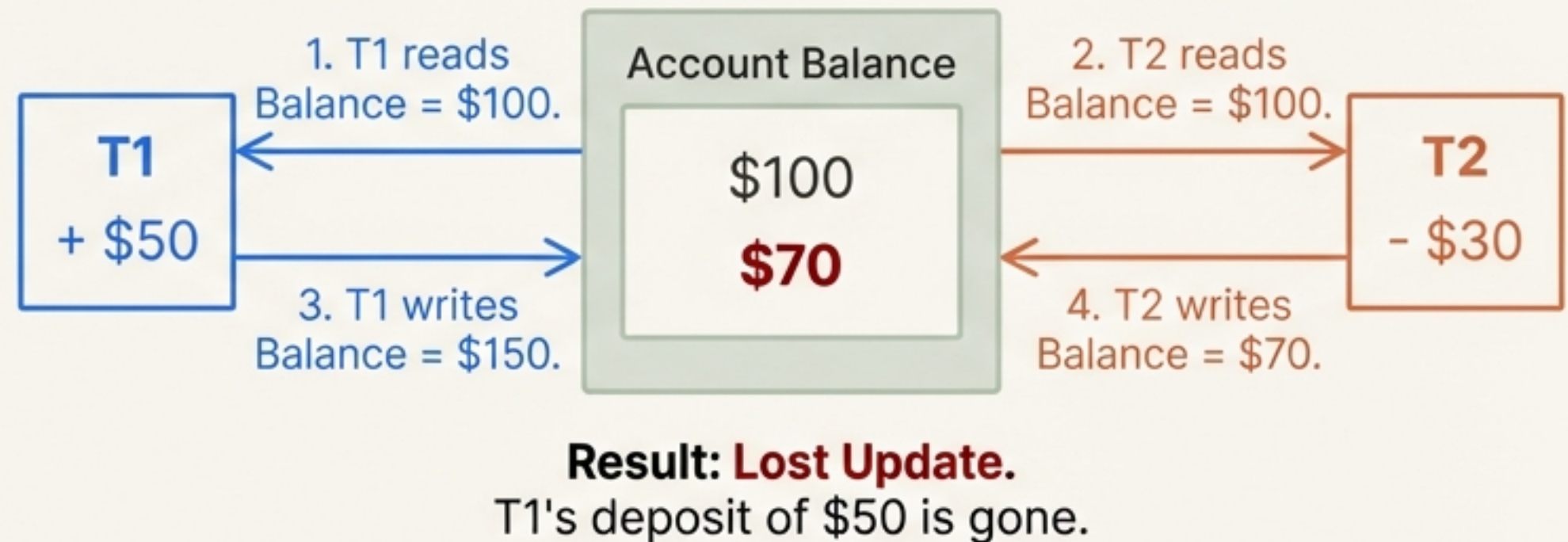
The Transaction:

An atomic unit of work. All of its operations succeed, or none do.

The Problem:

When multiple transactions access the same data simultaneously, their operations can **interleave** in ways that violate **data integrity**, leading to **incorrect results** (e.g., lost updates, dirty reads).

The “Lost Update” Problem



How do we allow **concurrent execution** for performance while **guaranteeing the outcome is the same as some serial execution**? This property is called **Serializability**.

The First Line of Defense: The Lock

What is a Lock?

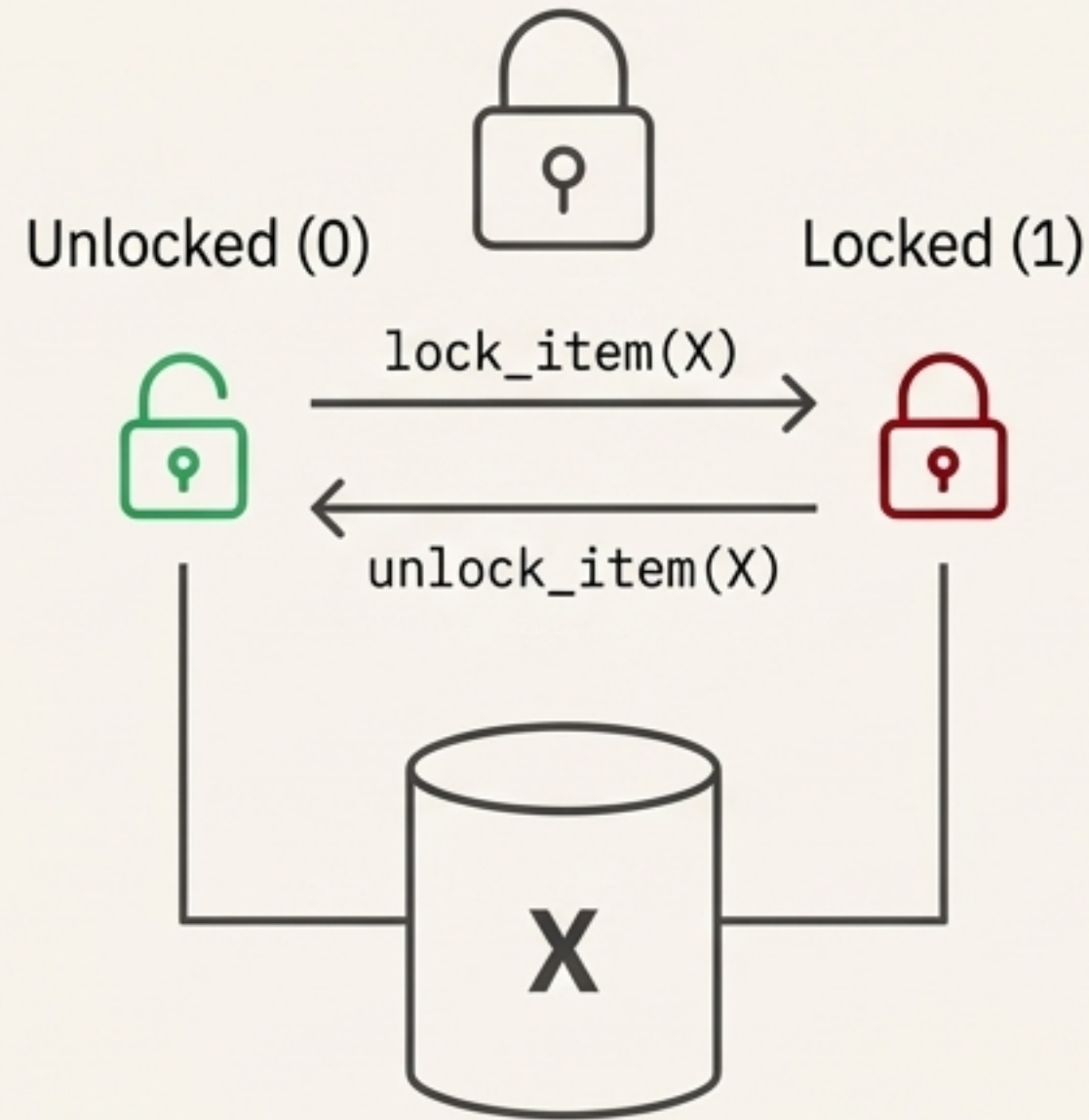
A variable associated with a data item. It describes the item's status for operations that can be applied.

Managed by a system component called the **Lock Manager**.

The Simplest Form: Binary Locks

A lock with two states:

- **Locked (1):** The item cannot be accessed by another transaction.
- **Unlocked (0):** The item is available.



The Basic Rules:

1. A transaction **must** issue `lock_item(X)` before any `read_item(X)` or `write_item(X)`.
2. It **must** issue `unlock_item(X)` after all operations on `X` are complete.
3. It will **not** try to lock an item it already has locked.
4. It will **not** try to unlock an item it does not hold a lock on.

Beyond Binary: Shared and Exclusive Locks

The Limitation of Binary Locks:

Binary locking is too restrictive. It prevents multiple transactions from even *reading* the same data item concurrently, even though read operations don't conflict with each other.

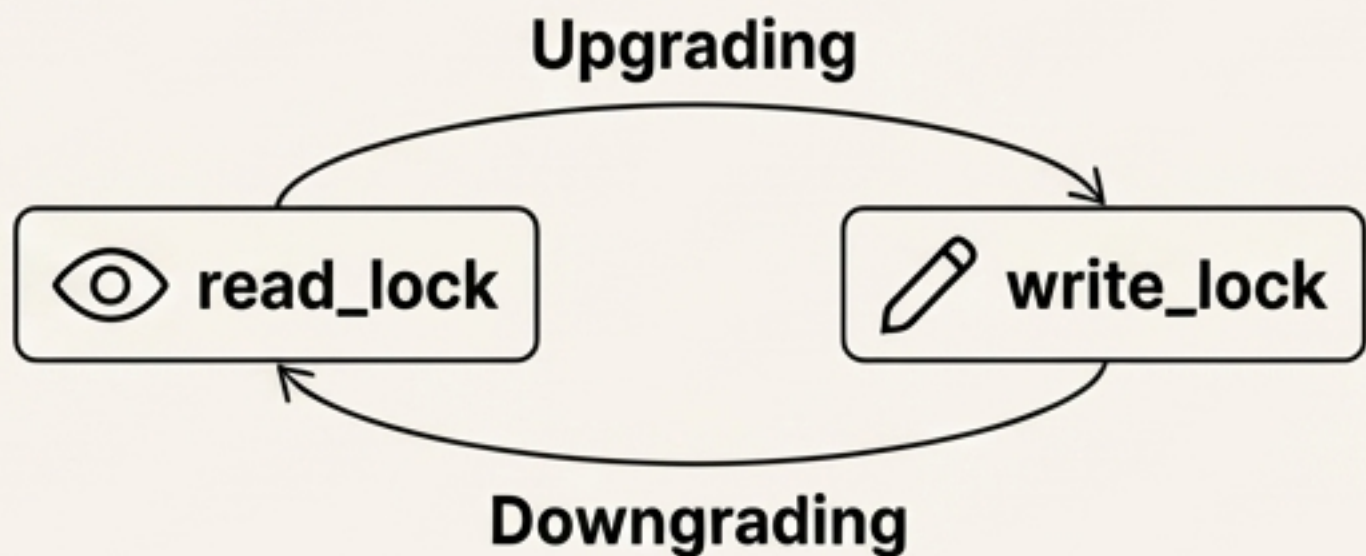
A More Granular System: Shared/Exclusive (or Read/Write) Locks.

- **Shared Lock (`read\_lock`)**: Multiple transactions can hold a shared lock on an item simultaneously. Used for reading.
- **Exclusive Lock (`write\_lock`)**: Only one transaction can hold an exclusive lock on an item. Required for writing.

Compatibility Matrix	Shared (Read)	Exclusive (Write)
Shared (Read)	✓ Yes	✗ No
Exclusive (Write)	✗ No	✗ No

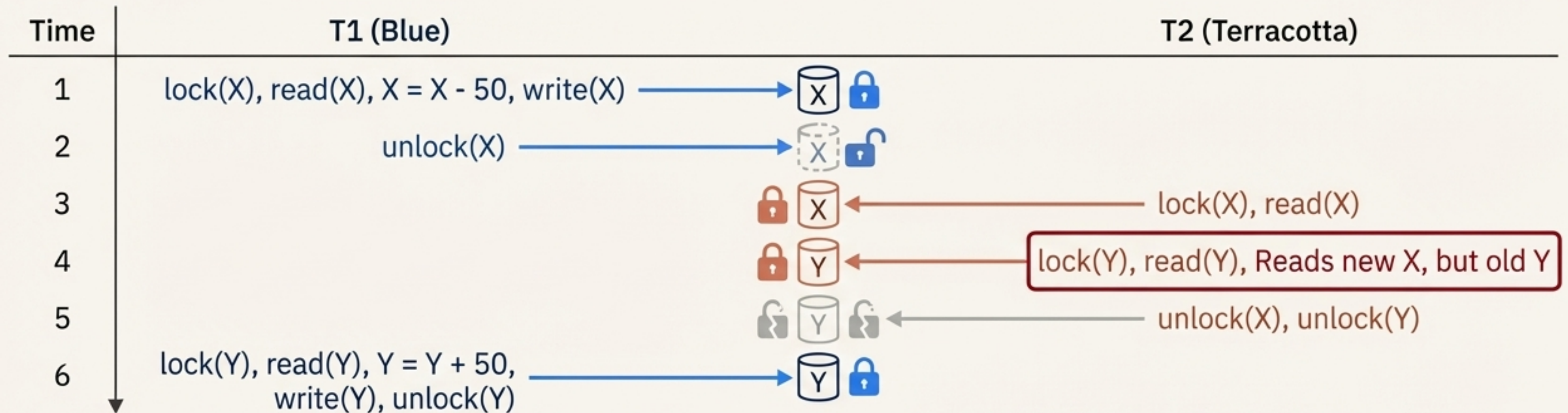
Lock Conversion:

A transaction holding a lock can convert it.



Why Simple Locking Fails to Guarantee Serializability

The Flaw: A transaction can acquire and release locks in an ad-hoc manner, allowing other transactions to interleave in non-serializable ways.



Analysis: This schedule uses locks correctly but is not serializable. T1's lock on X is released too early, allowing T2 to read an inconsistent state (new X but old Y) before T1 completes its full operation. This demonstrates the need for a protocol to govern *when* locks can be acquired and released.

Imposing Order: The Two-Phase Locking (2PL) Protocol

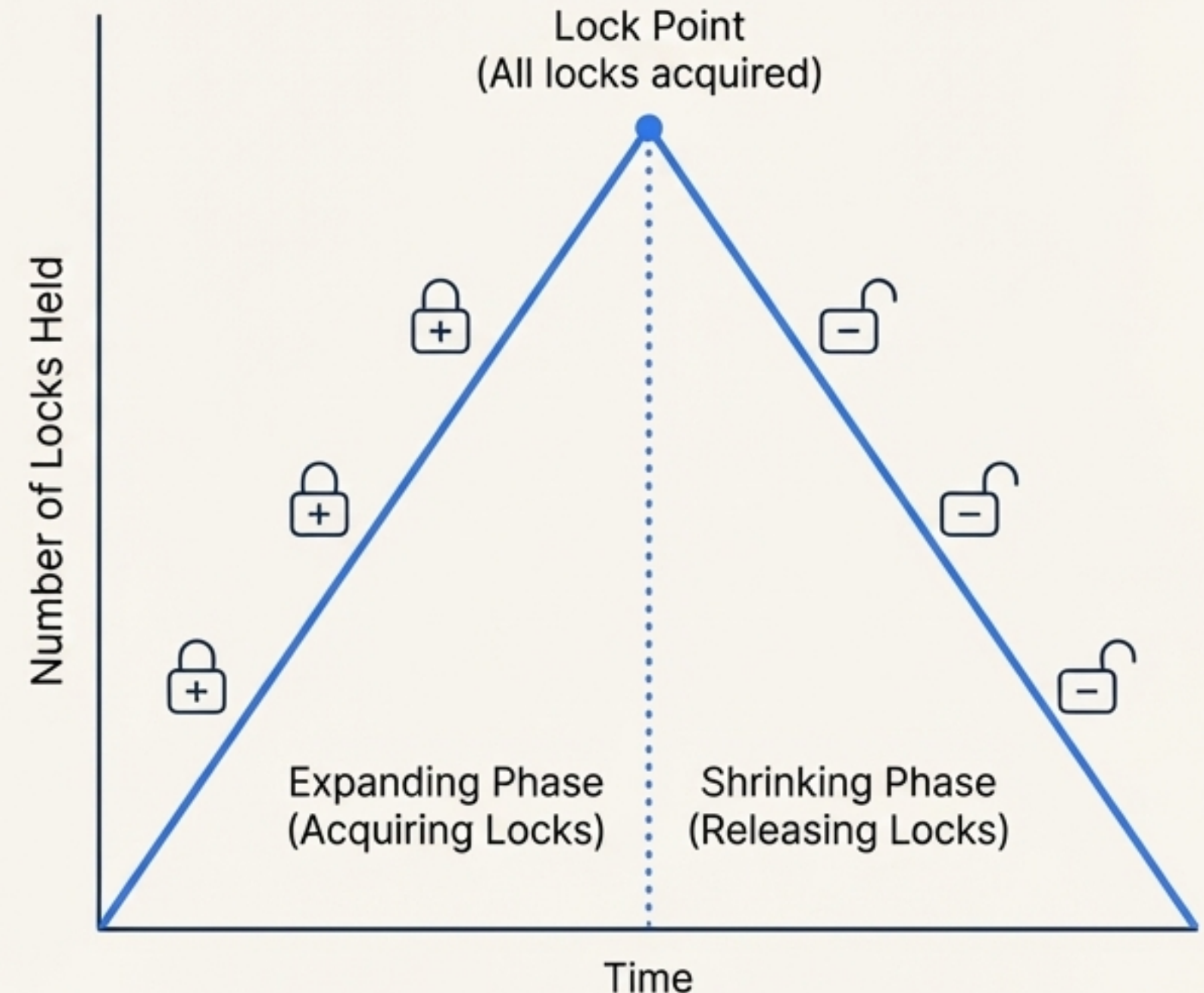
The Golden Rule of 2PL:

In any transaction, all locking operations must **precede the first unlock operation**. A transaction cannot obtain a new lock after it has released any lock.

The Two Phases:

1. **Expanding (or Growing) Phase:** The transaction acquires locks but does not release any. Lock upgrades (`read` to `write`) can happen here.
2. **Shrinking Phase:** The transaction releases its locks but cannot acquire any new ones. Lock downgrades (`write` to `read`) can happen here.

If every transaction in a schedule follows the 2PL protocol, the schedule is **guaranteed to be serializable**.



A Family of Protocols: Variations of 2PL

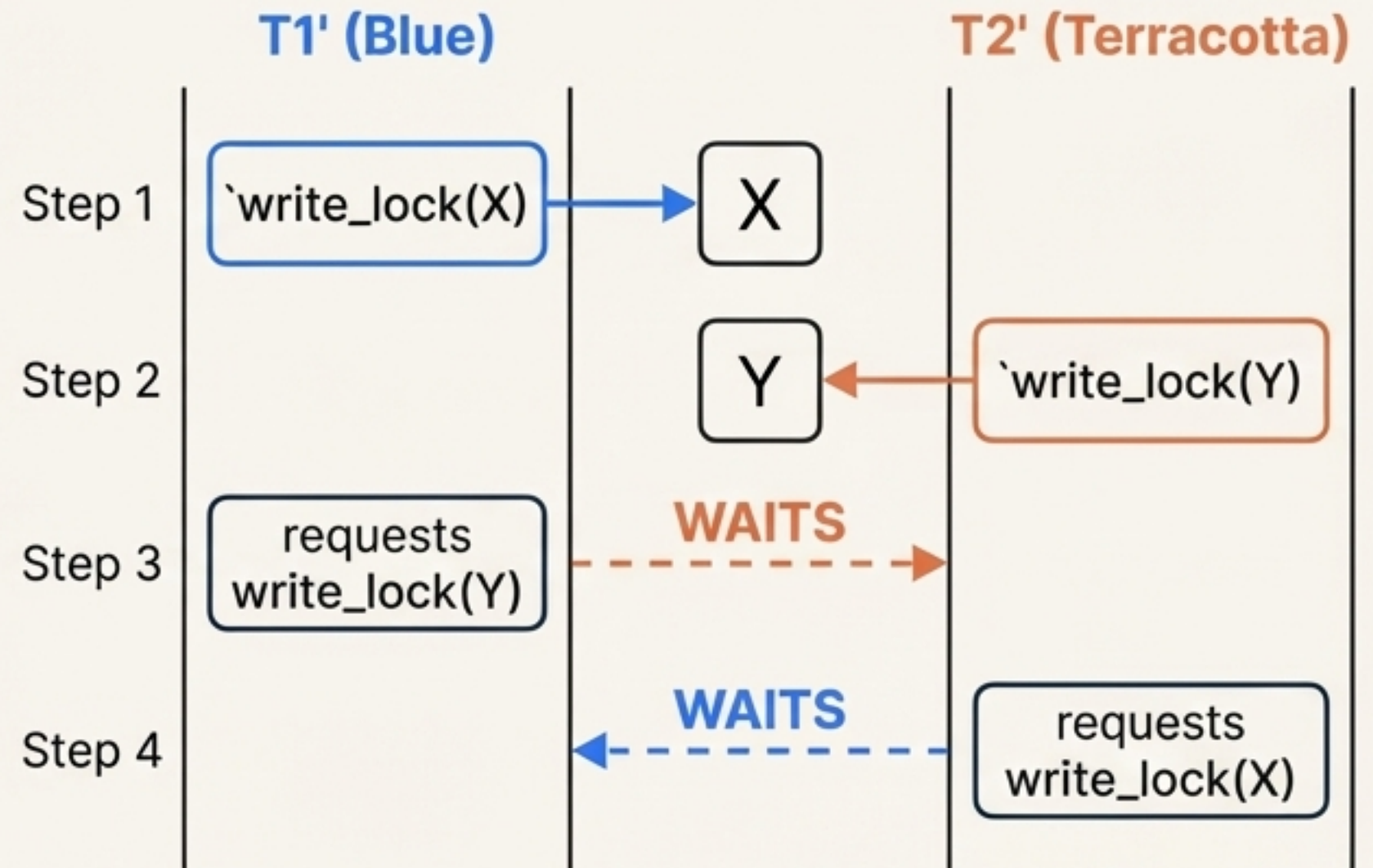
Protocol	Core Rule	Key Property
Basic 2PL	Follows the expanding/shrinking phase rule.	Guarantees serializability.
Conservative (Static) 2PL	Locks all required items before the transaction begins. (Pre-declaration of read/write sets). It's effectively always in its shrinking phase.	Deadlock-free , but can be impractical and limit concurrency.
Strict 2PL	Holds all exclusive (write) locks until after it commits or aborts.	Prevents 'dirty reads' and avoids cascading aborts. Highly practical.
Rigorous 2PL	Holds all locks (shared and exclusive) until after it commits or aborts.	Even stricter than Strict 2PL. Simplifies implementation.

The Unintended Consequence: Deadlock

Definition:

Deadlock occurs when each transaction in a set of two or more transactions is waiting for an item locked by another transaction in the set.

How a Deadlock is Created:



Outcome: A **circular dependency**. T1' is waiting for T2', and T2' is waiting for T1'. Neither can proceed. They are '**stuck**' indefinitely.

Visualizing the Impasse: The Wait-For Graph

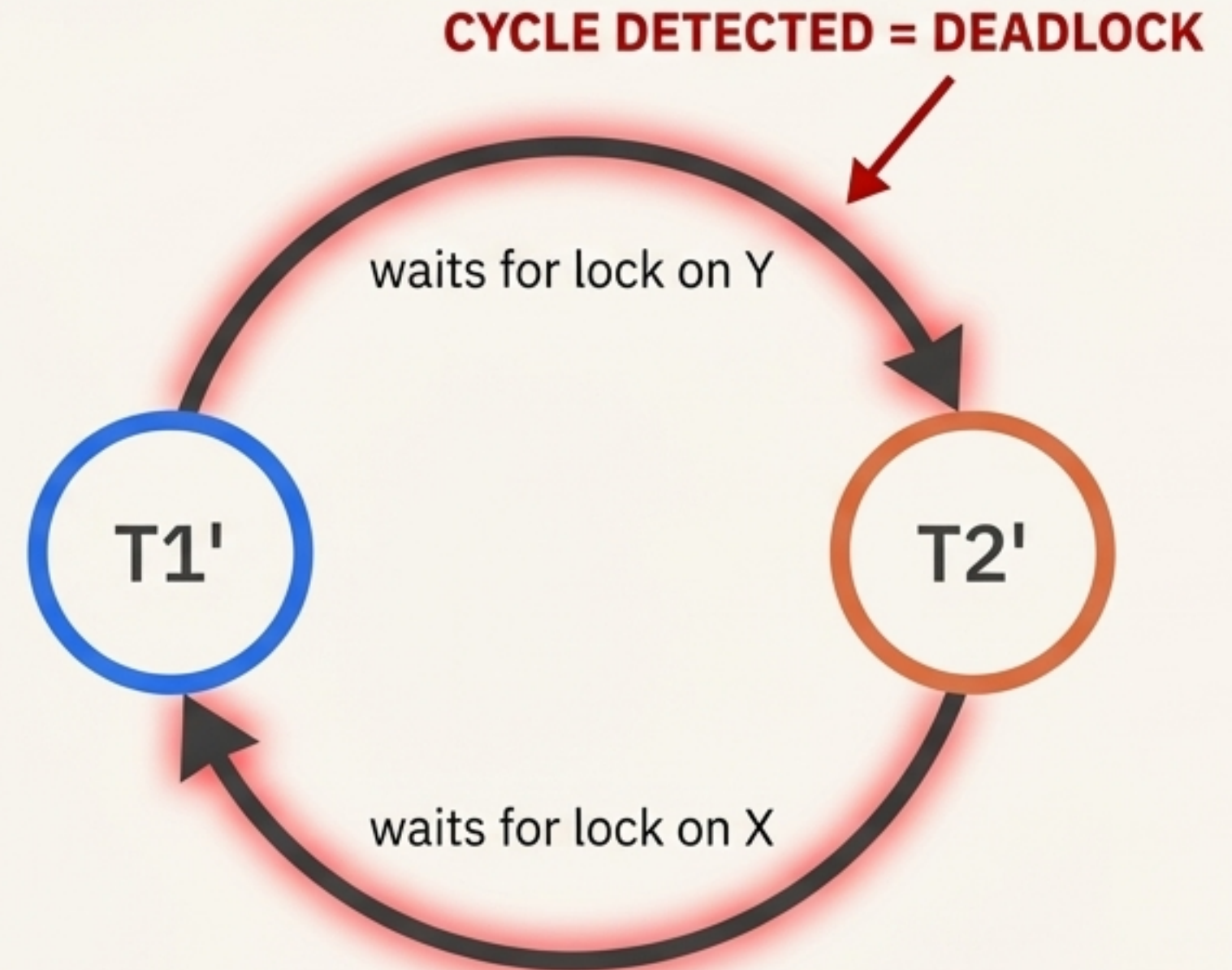
Purpose:

A graph used by the system to detect deadlock states.

How it Works:

- Nodes represent active transactions.
- A directed edge from T_i to T_j is created if transaction T_i is waiting to lock an item currently held by transaction T_j .

The **Rule for Detection**: A deadlock exists if and only if the wait-for graph contains a **cycle**.



Strategies for Managing Deadlock

Three Core Approaches:

1. Deadlock Prevention

Protocols that intrinsically prevent deadlocks from occurring.

Examples:

- **Conservative 2PL:** Lock all items upfront.
- **Item Ordering:** Acquire locks in a pre-defined global order (e.g., alphabetical).

Often impractical and can severely limit concurrency.

2. Deadlock Detection & Recovery

Allow deadlocks to happen, detect them, and then break them.

Detection: Use the wait-for graph to find cycles.

Recovery: Select a 'victim' transaction to abort and roll back, releasing its locks. Victim selection often targets younger transactions or those with fewer updates.

3. Deadlock Avoidance

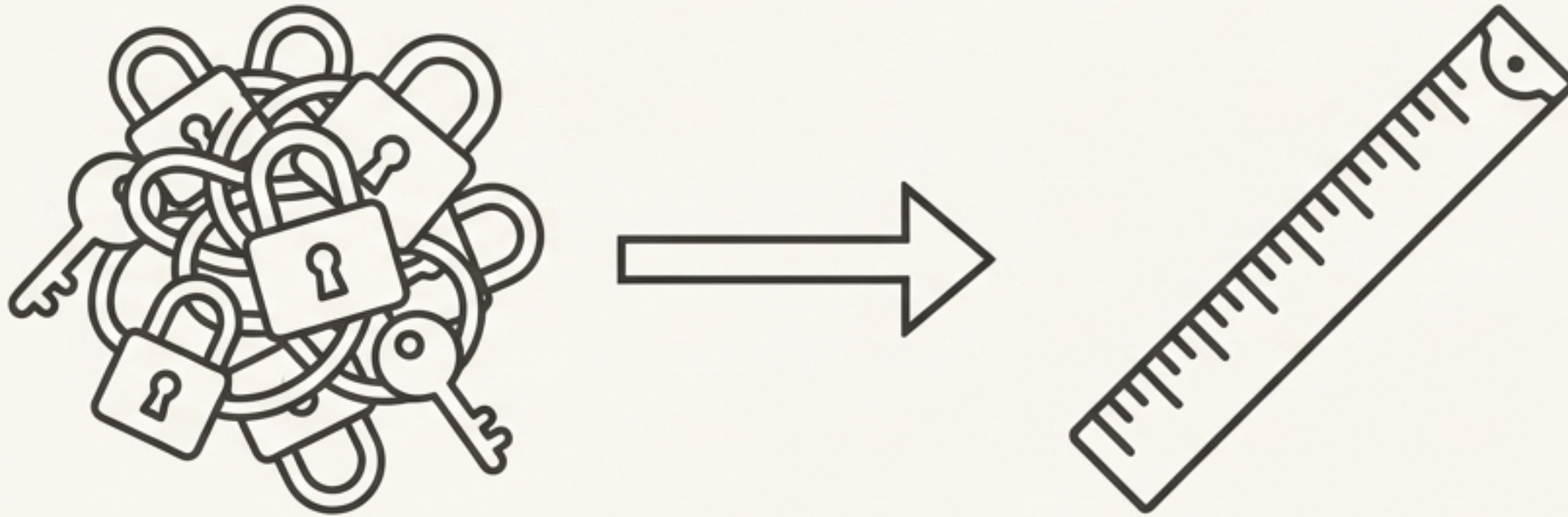
Use transaction timestamps to decide whether a transaction should wait or be rolled back, preventing circular waits.

Wait-Die: An older transaction can wait for a younger one. A younger transaction 'dies' (is aborted) if it requests a lock held by an older one.

Wound-Wait: An older transaction 'wounds' (aborts) a younger one to get a lock. A younger transaction can wait for an older one.

An Alternative Philosophy: Concurrency Control by Timestamp

Instead of using locks to control access, we can enforce a serial order on transactions based on when they started.



Key Advantage:

Timestamp-based protocols **do not use locks.**

Therefore, **deadlocks cannot occur.**

What is a Timestamp?

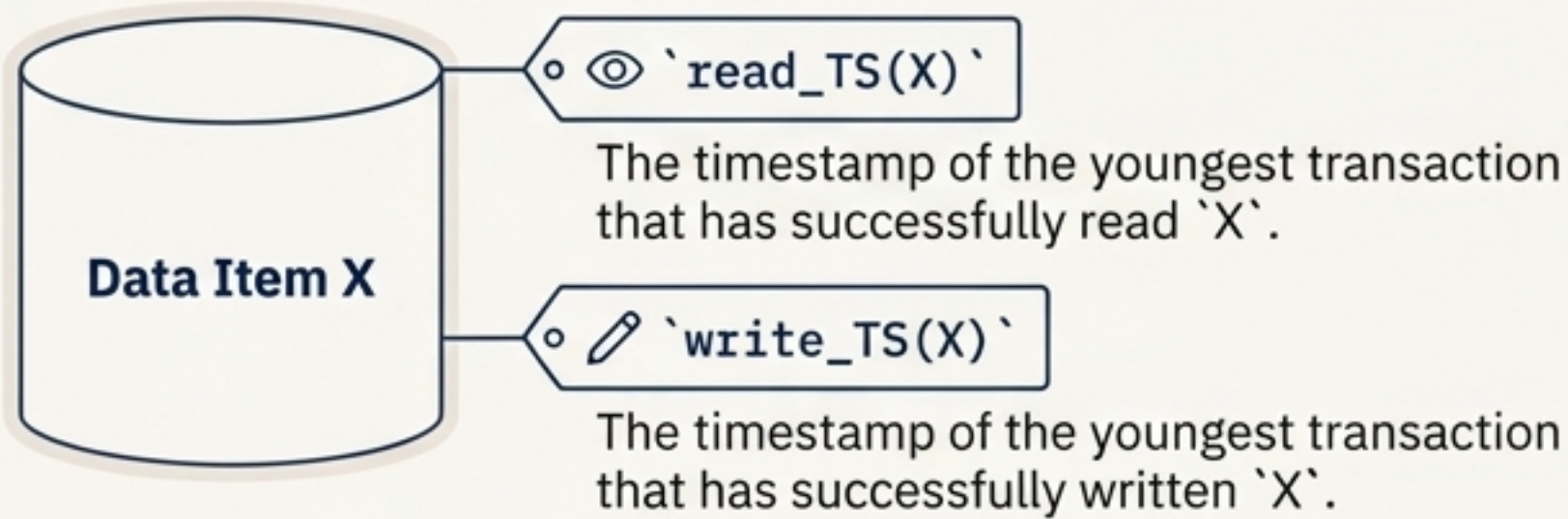
A unique identifier assigned to each transaction ($TS(T)$).

Typically assigned in the order transactions are submitted (e.g., from a system clock or an incrementing counter).

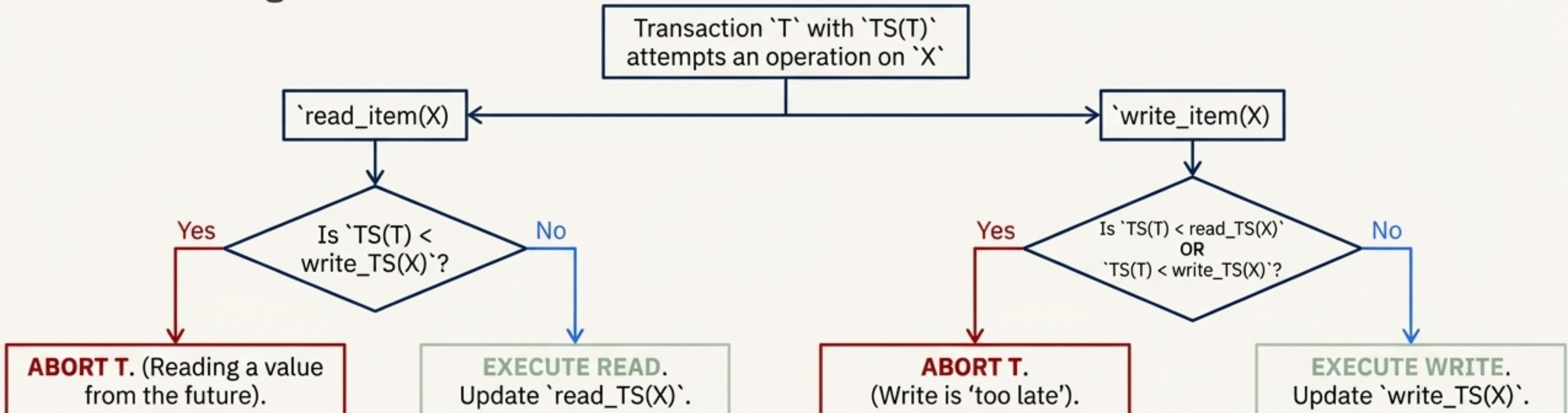
The Goal: Ensure that for any pair of conflicting operations, the one from the older transaction (with the smaller timestamp) is executed first.

The Mechanics of Timestamp Ordering (TO)

System Bookkeeping



The Basic TO Algorithm



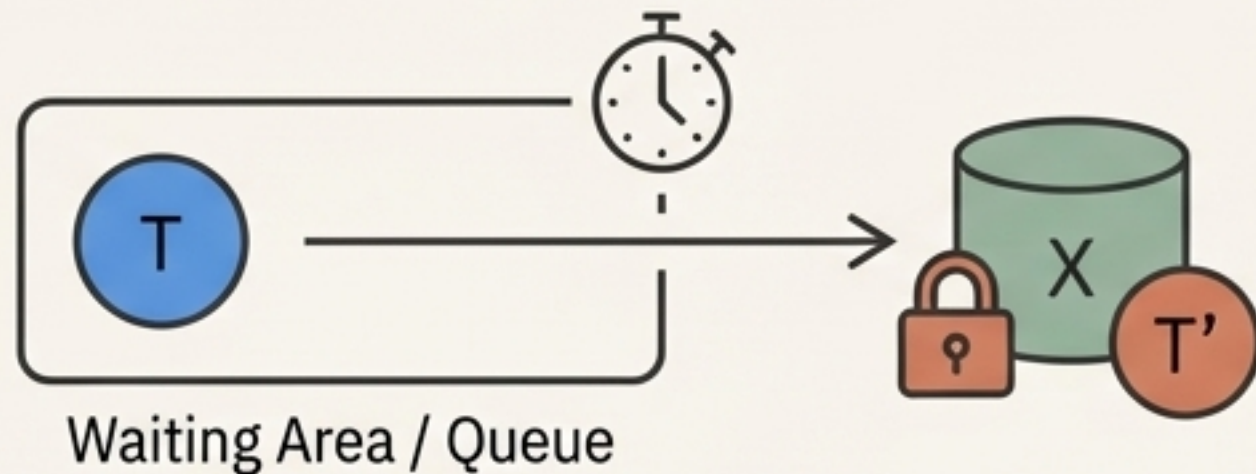
Refining the Protocol: Strict TO & Thomas's Write Rule

Challenge with Basic TO: Can produce schedules that are not “strict” (i.e., a transaction might read a value from an uncommitted transaction). This can also lead to starvation if a transaction is repeatedly aborted.

Refinement 1: Strict Timestamp Ordering

Rule: If a transaction T attempts to read or write an item X that has been written by a younger, uncommitted transaction T' , T 's operation is **delayed** until T' commits or aborts.

Benefit: Ensures schedules are both strict and conflict serializable, avoiding cascading aborts.

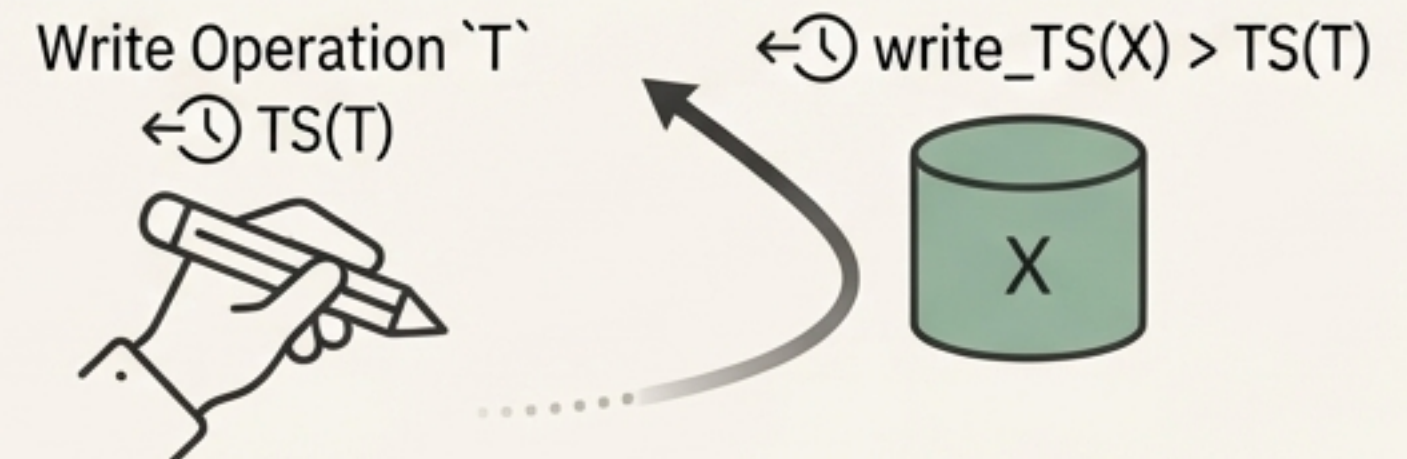


Refinement 2: Thomas's Write Rule

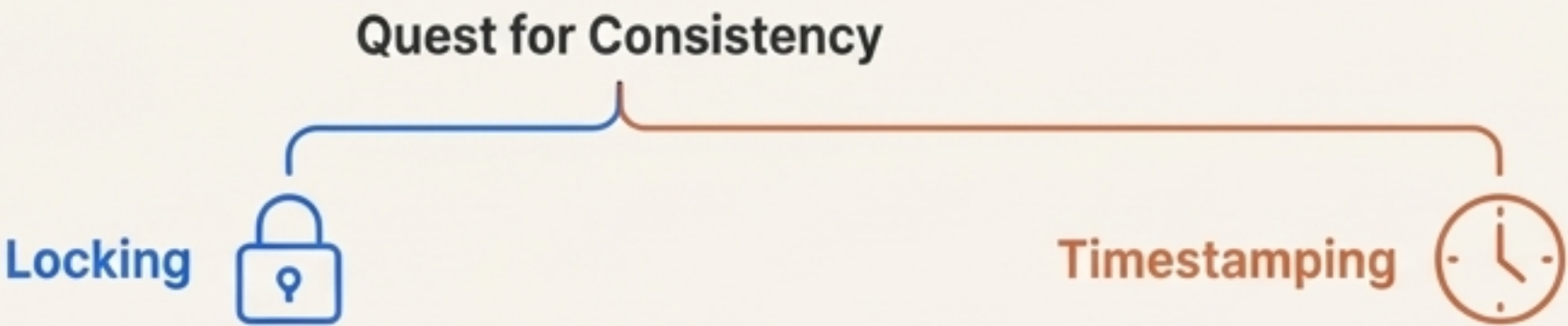
Rule: Modifies the basic TO check for $\text{write_item}(X)$. If $\text{TS}(T) < \text{write_TS}(X)$, instead of aborting T , the system simply **ignores** T 's write operation.

Rationale: The write is obsolete; a younger transaction has already produced a more current value.

Benefit: Rejects fewer write operations, potentially increasing concurrency. The resulting schedules are not necessarily conflict serializable but are still correct.



The Two Paths to Consistency: A Summary



Feature	Locking-Based (e.g., Strict 2PL)	Timestamp-Based (e.g., Basic TO)
Core Mechanism	Transactions request permission (locks) before accessing data. Conflicts cause waiting.	Transactions are ordered by timestamps. Conflicts with the established order cause aborts.
Deadlock	A significant possibility that requires explicit management (prevention, detection, etc.).	Not possible. The timestamp ordering prevents circular waits.
Primary Penalty	Waiting. Transactions are often blocked, which can reduce concurrency.	Aborts & Restarts. Transactions may be rolled back frequently, causing wasted work.
Conflict Type	Detects conflicts dynamically as they occur (when a lock is requested on a locked item).	Assumes conflicts are frequent and enforces a serial order from the start.
Overhead	Managing a lock table and wait queues.	Storing read/write timestamps for every data item and managing timestamps.

The choice between locking and timestamping is a fundamental trade-off between waiting and restarting. Modern systems often use sophisticated hybrid approaches, but understanding these two foundational models is key to mastering database concurrency.